

PROPLEDGER ARCHITECTURE SERIES — VOLUME 05 OF 10

Transactions & Concurrency

ACID Guarantees, Row-Level Locking & Race Condition Mitigation

DOMAIN LEVEL:
Enterprise Real Estate SaaS

Comparable System: Yardi Voyager / RealPage

FRAMEWORKS:
Spring Boot 3.3 | PG 16 | React 19

PAGE BUDGET:
10 Pages Complete Reference

1. DOCUMENT EXECUTIVE ABSTRACT & SCOPE

This volume establishes the transactional reliability and concurrency control architecture of PropLedge. Designed for mission-critical financial subledgers, it provides an in-depth breakdown of PostgreSQL MVCC isolation levels, Spring Boot @Transactional AOP proxy mechanics, pessimistic write locking (SELECT FOR UPDATE), optimistic JPA versioning (@Version), and mathematical deadlock elimination algorithms.

2. VOLUME SYLLABUS & CORE CAPABILITIES

Page	Chapter Title	Primary Technical Focus & Learning Outcome
02	Multi-Table Transaction Workflows	Atomic boundaries across payments, invoices, leases, and units
03	Isolation Levels & Anomalies	Dirty reads, non-repeatable reads, phantom reads, and write skew
04	Real Estate Concurrency Hazards	Race conditions in payment posting and concurrent lease bookings
05	Spring Boot @Transactional Internals	AOP CGLIB proxies, rollback rules, and propagation levels
06	The Proxy Self-Invocation Trap	Why calling this.method() bypasses transactions and the fix
07	Pessimistic Row Locking	SELECT FOR UPDATE, NOWAIT, and lock acquisition timeouts
08	Optimistic Locking (@Version)	Preventing lost updates on work order notes across field tablets
09	Mathematical Deadlock Prevention	Resource ordering discipline: locking IDs in strict ascending sequence
10	Concurrency & ACID Interview Q&A	Senior engineering interview scenarios and failure recovery

SENIOR STAFF ENGINEER REVIEWER GUIDANCE

This volume is engineered specifically for software engineering and database architecture interviews at Tier-1 real estate technology and enterprise SaaS corporations (Yardi, RealPage, AppFolio, CoStar). Every code sample reflects active, compilable implementations from the PropLedge production repository.

2. MULTI-TABLE TRANSACTION WORKFLOWS & ACID BOUNDARIES

ATOMIC MULTI-ENTITY STATE MUTATIONS

1. The Financial Payment Allocation Workflow

A resident pays \$2,500 via ACH to settle an overdue invoice (\$1,800) and a current rent invoice (\$700). This operation touches four tables and must execute as a single atomic transaction:

- 1. Insert Payment Voucher: Records \$2,500 receipt in 'payments' table with SETTLED status.
- 2. Insert Allocations: Inserts two rows in 'payment_allocations' (\$1,800 to Inv #1, \$700 to Inv #2).
- 3. Recalculate Invoice Balances: Trigger recalculates balance_due = \$0 on both invoices, updating status to 'PAID'.
- 4. Audit Log Persistence: Records immutable audit entry detailing who applied the funds.

2. Production Code: Multi-Invoice Payment Allocation

```
// CODE LISTING: SQL – ATOMIC MULTI-INVOICE SETTLEMENT TRANSACTION
```

```
-- database/transactions/transactions.sql
BEGIN;

-- Step 1: Acquire exclusive row locks on invoices in ascending order
SELECT id, total_amount, balance_due, status
FROM invoices
WHERE id IN (1, 2)
ORDER BY id ASC
FOR UPDATE;

-- Step 2: Record payment voucher
INSERT INTO payments (payment_reference, tenant_id, amount, payment_method, status, payment_date)
VALUES ('PAY-ACH-2026-0091', 1, 2500.00, 'ACH', 'SETTLED', NOW());

-- Step 3 & 4: Insert allocations (triggers maintain invoice balance_due)
INSERT INTO payment_allocations (payment_id, invoice_id, allocated_amount)
VALUES (currval('payments_id_seq'), 1, 1800.00);
```

3. POSTGRESQL ISOLATION LEVELS & ANOMALIES MATRIX

MULTI-VERSION CONCURRENCY CONTROL (MVCC) MECHANICS

1. ANSI SQL Isolation Levels & Concurrency Phenomena

PostgreSQL implements Multi-Version Concurrency Control (MVCC), where readers never block writers and writers never block readers:

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Write Skew (Anomaly)
Read Uncommitted	Prevented in PG	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible	Possible
Repeatable Read	Prevented	Prevented	Prevented (via Snapshot)	Possible
Serializable (SSI)	Prevented	Prevented	Prevented	Prevented

2. Why PostgreSQL 'Read Committed' is the Default

In 'Read Committed', each statement sees a snapshot of data committed before that specific statement began. It offers maximum throughput for 90% of web transactions while preventing dirty reads.

3. When to Escalate to 'Serializable'

In PropLedger's 'LeaseServiceImpl', contract signing escalates to 'Isolation.SERIALIZABLE'. This eliminates phantom reads and write skew when creating lease extensions or updating tenant lease pointers.

4. REAL ESTATE CONCURRENCY HAZARDS & RACE CONDITIONS

DOUBLE-BOOKING, OVER-ALLOCATION & WRITE SKEW

1. Hazard 1: Concurrent Payment Over-Allocation

Problem: An invoice has `balance_due` = \$1,000. At 10:00:00 AM, the tenant's auto-pay posts \$1,000. At the same second, the tenant manually clicks 'Pay Now' on the web portal. If both threads read `balance_due` = \$1,000 concurrently, both insert \$1,000 allocations, resulting in a -\$1,000 negative balance!

PropLedger Mitigation: Pessimistic write locking ('findByIdForUpdate') serializes both threads. Thread 2 waits for Thread 1 to commit, sees `balance_due` = \$0, and cleanly rejects the duplicate payment.

2. Hazard 2: The Double-Booking Race Condition

Problem: Two leasing agents create active leases on the same unit for overlapping dates simultaneously.

PropLedger Mitigation: Enforced at the engine layer via 'btree_gist' exclusion constraints. Even under multi-threaded load, PostgreSQL rejects the second transaction with error code 23P01.

3. Hazard 3: Lost Updates on Maintenance Notes

Problem: A property manager modifies vendor instructions on a work order while a technician in the basement updates labor hours from a mobile tablet. The manager's notes are overwritten!

PropLedger Mitigation: Optimistic locking (@Version) detects version mismatch and returns HTTP 409 Conflict.

5. SPRING BOOT @TRANSACTIONAL INTERNALS

AOP PROXY MECHANICS, PROPAGATION & ROLLBACK POLICIES

1. How @Transactional Works Under the Hood

Spring wraps transactional beans in a CGLIB dynamic proxy (TransactionInterceptor):

- 1. Method Interception: When a client calls paymentService.processPayment(), the CGLIB proxy intercepts the invocation.
- 2. Connection Acquisition: The proxy borrows a physical connection from HikariCP and sets connection.setAutoCommit(false).
- 3. Thread-Binding: Binds the connection to TransactionSynchronizationManager using a ThreadLocal.
- 4. Execution & Commit: Invokes the target method. On return, executes connection.commit() and closes connection.

2. Propagation Levels in PropLedger

Propagation Level	Behavior if Existing Transaction Exists	Primary PropLedger Use Case
REQUIRED (Default)	Joins existing transaction; creates one if none exists	Standard service methods (createProperty, createUnit)
REQUIRES_NEW	Suspends outer transaction; creates new independent TX	Audit logging (AuditLogService.logAction)
MANDATORY	Throws TransactionRequiredException if no TX exists	Low-level subledger allocation helper methods

3. Rollback Rule Gotcha: 'rollbackFor = Exception.class'

By default, Spring rolls back ONLY on unchecked exceptions (RuntimeException and Error). Checked exceptions (SQLException, IOException, BusinessException) do NOT trigger a rollback unless explicitly declared: '@Transactional(rollbackFor = Exception.class)'.

6. THE SPRING PROXY SELF-INVOCATION TRAP

WHY THIS.METHOD() BYPASSES TRANSACTIONS AND HOW TO FIX IT

1. The Self-Invocation Failure Mode

A classic senior engineering interview question: What happens if an un-annotated method calls a `@Transactional` method inside the same class?

// CODE LISTING: JAVA 21 – THE SELF-INVOCATION BUG

// ■ THE PROXY SELF-INVOCATION TRAP:

`@Service`

`public class LeaseServiceImpl implements LeaseService {`

`public void executeLeaseWorkflow(Long leaseId) {`

`// Internal call using 'this' pointer:`

`this.activateLeaseInternal(leaseId); // BUG: @Transactional is completely IGNORED!`

`}`

`@Transactional(isolation = Isolation.SERIALIZABLE)`

`public void activateLeaseInternal(Long leaseId) {`

`// Database modifications run with autoCommit = true! Zero transaction started!`

`}`

`}`

2. Why It Fails Architecturally

The 'this' pointer references the raw target Java object in memory, completely bypassing the dynamic Spring CGLIB proxy. Because the proxy is bypassed, 'TransactionInterceptor' never intercepts the call!

3. The Architectural Solution

- Solution 1 (Recommended): Extract the transactional method into a separate collaborator service bean (e.g. 'LeaseTransactionCoordinator').
- Solution 2: Inject the bean into itself via '@Lazy private LeaseService self;' and call 'self.activateLeaseInternal()'.

7. PESSIMISTIC ROW LOCKING & TIMEOUT MECHANICS

SELECT FOR UPDATE, NOWAIT & SKIP LOCKED

1. Pessimistic Write Lock Mechanics

In financial operations where conflicting writes are unacceptable, pessimistic locking acquires an exclusive row lock at the database engine layer for the duration of the transaction:

```
// CODE LISTING: JAVA 21 – PESSIMISTIC LOCK REPOSITORY IMPLEMENTATION

// InvoiceRepository.java: Pessimistic Write Lock
public interface InvoiceRepository extends JpaRepository<Invoice, Long> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @QueryHints({@QueryHint(name = "jakarta.persistence.lock.timeout", value = "3000")})
    @Query("SELECT i FROM Invoice i WHERE i.id = :id")
    Optional<Invoice> findByIdForUpdate(@Param("id") Long id);
}

// Generated SQL executed against PostgreSQL 16:
// SELECT id, total_amount, balance_due FROM invoices WHERE id = 104 FOR UPDATE;
```

2. Lock Modifiers Compared

SQL Modifier	Behavior on Contention	Primary PropLedger Use Case
FOR UPDATE	Blocks waiting thread until holding transaction commits	Single invoice payment allocation
FOR UPDATE NOWAIT	Immediately throws lock_not_available error	Interactive UI checkout (fails fast if locked)
FOR UPDATE SKIP LOCKED	Skips locked rows; returns only unlocked records	High-throughput worker queues (batch billing jobs)

8. OPTIMISTIC LOCKING (@VERSION) & LOST UPDATE DEFENSE

JPA VERSIONING, STALE UPDATES & HTTP 409 CONFLICT

1. Optimistic Locking Philosophy

Unlike pessimistic locking which holds database locks, optimistic locking assumes conflicts are rare. An integer 'version' column is incremented on every update:

```
// CODE LISTING: JAVA 21 – JPA @VERSION IMPLEMENTATION

// WorkOrder.java: Optimistic Versioning Entity
@Entity
@Table(name = "work_orders")
@Getter @Setter
public class WorkOrder {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String description;
    private BigDecimal laborCost;
    private BigDecimal materialCost;

    @Version // Hibernate automatically manages this version counter
    @Column(name = "version")
    private Long version;
}
```

2. Under the Hood SQL & Conflict Detection

When saving: 'UPDATE work_orders SET labor_cost = 150.00, version = 2 WHERE id = 42 AND version = 1'. If another user saved first, rows affected is 0! Hibernate throws 'OptimisticLockException', caught by GlobalExceptionHandler and mapped to HTTP 409 Conflict:

```
// CODE LISTING: JAVA 21 – HTTP 409 CONFLICT HANDLER

// GlobalExceptionHandler.java: Handling Optimistic Lock Collisions
@ExceptionHandler(OptimisticLockingFailureException.class)
public ResponseEntity<ErrorResponse> handleOptimisticLock(OptimisticLockingFailureException ex) {
    return ResponseEntity.status(HttpStatus.CONFLICT).body(
        ErrorResponse.builder()
            .status(409)
            .error("Conflict")
            .message("The record was modified by another user. Please refresh and review latest updates.")
            .build()
    );
}
```

9. MATHEMATICAL DEADLOCK ELIMINATION

RESOURCE ORDERING ALGORITHM & LOCK GRAPH CYCLES

1. Anatomy of a Deadlock in Multi-Resource Settlement

A deadlock occurs when two concurrent transactions attempt to lock the same resources in opposite orders:

- Transaction 1: Locks Invoice #101, then attempts to lock Invoice #102.
- Transaction 2: Locks Invoice #102, then attempts to lock Invoice #101.
- Result: Both transactions wait indefinitely. After 1 second (deadlock_timeout), PostgreSQL aborts one transaction with error 40P01.

2. The PropLedger Resource Ordering Algorithm

PropLedger eliminates deadlocks mathematically by enforcing **Ascending ID Resource Ordering** before acquiring any locks:

// CODE LISTING: JAVA 21 – DEADLOCK-FREE RESOURCE ORDERING

```
// PaymentServiceImpl.java: Strict Resource Ordering
@Transactional(isolation = Isolation.READ_COMMITTED, rollbackFor = Exception.class)
public PaymentResponseDto processPayment(PaymentRequestDto dto) {
    // 1. SORT INVOICE IDs IN ASCENDING ORDER:
    List<Long> sortedInvoiceIds = dto.getAllocations().stream()
        .map(AllocationItemDto::getInvoiceId)
        .sorted() // GUARANTEES STRICT RESOURCE ORDERING!
        .collect(Collectors.toList());

    // 2. ACQUIRE LOCKS SEQUENTIALLY:
    Map<Long, Invoice> lockedInvoices = new HashMap<>();
    for (Long invId : sortedInvoiceIds) {
        Invoice invoice = invoiceRepository.findByIdForUpdate(invId)
            .orElseThrow(() -> new EntityNotFoundException("Invoice: " + invId));
        lockedInvoices.put(invId, invoice);
    }
    // All transactions request locks in identical order: (101 -> 102). Cycles are IMPOSSIBLE!
}
```

10. CONCURRENCY CONTROL & ACID INTERVIEW Q&A

SENIOR SYSTEMS DESIGN & BACKEND INTERVIEW SCENARIOS

1. High-Yield Interview Scenarios

Question 1: When should you use Optimistic Locking vs Pessimistic Locking?

Answer: Use Optimistic Locking when read frequency is high, write contention is low, and human latency is involved (e.g. property metadata, work order notes). Use Pessimistic Locking when write contention is high, conflicts are unacceptable, and financial balances are being calculated (e.g. invoice payment allocations).

Question 2: How does Write-Ahead Logging (WAL) guarantee Durability (D in ACID)?

Answer: When a transaction commits, modifications in shared_buffers RAM are written sequentially to the Write-Ahead Log (WAL) on disk before reporting success to the client. If power fails immediately after commit, PostgreSQL replays the WAL log on recovery, reconstructing committed state into data pages.

Question 3: How do you handle distributed transactions across independent microservices?

Answer: Avoid 2-Phase Commit (2PC) due to high latency and coordinator failure risks. Instead, implement the Saga Pattern with choreographed or orchestrated events (via Kafka/RabbitMQ) and compensating transactions (e.g. if payment capture fails, dispatch a 'VoidInvoiceCommand' compensation event).

SENIOR STAFF ENGINEER INTERVIEW TIP

Always articulate deadlock elimination via resource ordering. It proves you understand distributed computing and database engine lock graphs.